

F&B; NEXUS

Kiến trúc ứng dụng, dữ liệu và triển khai

Quyết định kiến trúc cuối cùng cho nền tảng SaaS đa tenant có khả năng tùy biến quy trình vận hành theo từng chuỗi F&B.;

Quyết định chốt: một monorepo và một bộ sản phẩm dùng chung; tùy biến bằng cấu hình, rule, workflow và extension; dữ liệu mặc định nằm trong PostgreSQL pooled với business_id + RLS; triển khai một pool cell trước, thêm dedicated cell khi có nhu cầu thực tế.

Ba nguyên tắc dẫn đường

- Tùy biến nghiệp vụ và cách ly hạ tầng là hai trục độc lập. Quán có quy trình riêng vẫn có thể dùng chung ứng dụng và database.
- Mọi cell chạy cùng tập image và cùng release train. Không fork codebase, không giữ riêng phiên bản cũ cho từng tenant.
- Chỉ trả chi phí vận hành của nhiều cell khi số liệu tải, hợp đồng hoặc yêu cầu phức hồi chứng minh rằng điều đó cần thiết.

Phạm vi

Tài liệu này chốt kiến trúc ứng dụng, mô hình tùy biến, trục dữ liệu, kiến trúc control plane/cell, luồng dữ liệu, phương án deploy theo giai đoạn và các điều kiện mở rộng. Nội dung được đối chiếu với cấu trúc hiện tại của repo fnb và các ADR đã có.

1. Kiến trúc ứng dụng

Giữ Service-Based Architecture hiện tại. Mỗi domain service là một deployable độc lập, nhưng toàn bộ backend Java vẫn ở trong một monorepo để kiểm soát dependency, migration và thay đổi xuyên service.

Trục	Quyết định
Codebase	Một monorepo Java dùng chung; không tạo repo hoặc nhánh riêng cho từng quán.
Backend	Các service theo domain: identity, platform, files, notify, integration, cashclose, workforce, inventory, reporting và gateway.
Bên trong service	Layered: api → service → repository; chỉ mở extension ở các điểm nghiệp vụ được chọn.
Frontend	Mobile và console dùng chung cho mọi tenant; UI thay đổi theo quyền, workflow và field definition.
Deployment	Mỗi service tạo một image riêng; cùng image được triển khai ở mọi cell.
Database	Mỗi cell có một OLTP database, chia schema theo domain.

Thang tùy biến

Tầng	Cơ chế	Ví dụ
1	Config	Ngưỡng chênh lệch, mệnh giá, SLA.
2	Rule / policy	Thiếu trên 200.000 ở ca tối thì chủ quán duyệt và phải có ảnh.
3	Workflow-as-data	Đếm mù, hai người đếm, sau đó đối soát.
4	Extension field	Số niêm phong túi tiền lưu trong ext JSONB và validate bằng schema.
5	Code extension	Công thức phân bổ thiếu hụt theo giờ công.
6	Out-of-process	Đẩy phiếu đã duyệt sang MISA hoặc hệ thống riêng của khách.

Luôn giải quyết yêu cầu ở tầng thấp nhất đủ dùng. Giá trị cạnh tranh nằm ở rule, workflow và trường mở rộng; code riêng là ngoại lệ có phí.

2. Ranh giới giữa core và phần tùy biến

Core giữ các invariant tài chính và bảo mật. Extension được thay đổi policy hoặc bổ sung hành vi, nhưng không được đi vòng qua các invariant này.

Core bắt buộc giữ	Tenant được phép tùy biến
Ledger chỉ append; sổ quỹ phải cân; freeze số liệu cuối khi duyệt.	Ai duyệt, duyệt ở ngưỡng nào, cần bằng chứng gì.
Audit log và lịch sử quyết định không bị ghi đè.	Số bước, vai trò thực hiện, điều kiện chuyển bước.
business_id, RLS, phân quyền và write ownership.	Trường nhập bổ sung và validation tương ứng.
Model chuẩn phục vụ báo cáo, dashboard và AI.	Mã nội bộ của quán, nhưng phải map về taxonomy chuẩn.

Ví dụ xuyên suốt

	Quán A	Quán B
Đếm tiền	Một nhân viên đếm.	Đếm mù; nhân viên và ca trưởng đếm độc lập.
Duyệt	Quản lý duyệt nếu lệch trên 50.000.	Chủ quán duyệt nếu thiếu trên 200.000 ở ca tối; bắt buộc ảnh.
Dữ liệu thêm	Không có.	Số niêm phong túi nộp ngân hàng.
Logic đặc biệt	Default extension không làm gì.	Phân bổ thiếu hụt theo giờ công sau khi core đã freeze số cuối.

Hai quán vẫn đi qua cùng cashclose-service và cùng model cash_close. Sự khác biệt nằm trong rule, workflow definition, field definition và extension binding. Không rải điều kiện if businessId == X trong core.

Nếu tenant muốn đổi database

- Thêm vài trường: dùng ext JSONB + schema validation.
- Thêm quan hệ hoặc tập bản ghi: tạo bảng theo tính năng, có business_id + RLS; migration nằm trong bộ migration chung.
- Thay đổi grain hoặc ý nghĩa của bảng lỗi: tổng quát hóa model cho toàn hệ thống, tạo khái niệm nghiệp vụ mới, hoặc tạm từ chối. Không sửa ý nghĩa bảng lỗi chỉ cho một tenant.

3. Trục dữ liệu

Chọn pooled shared schema + RLS làm mặc định. Dedicated database là một tier vận hành dành cho tenant có yêu cầu cách ly, phục hồi hoặc quy mô rõ ràng. Không dùng schema-per-tenant.

Cấu trúc của một pool cell

fnbx_oltp

identity · platform · files · notify ·
integration · cashclose · workforce ·
inventory · analytics · ext_

Mô hình	Dùng khi nào	Đánh giá
Pooled	Mặc định cho đa số tenant, kể cả tenant có workflow riêng.	Chi phí thấp, migration một lần, analytics thuận lợi.
Schema-per-tenant	Không chọn.	Cách ly tài nguyên yếu; migration và catalog tăng theo số tenant; dễ phát sinh lỗi search_path.
Dedicated DB/cell	Hợp đồng yêu cầu, noisy neighbor đo được, restore/PITR riêng hoặc data residency.	Chi phí cao hơn; migration và giám sát phải chạy theo cell.

Các lớp cách ly bắt buộc

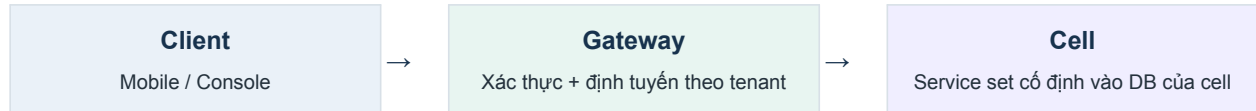
- business_id lấy từ danh tính đã xác thực, không tin giá trị do client tự truyền.
- Đặt tenant context ở đầu mỗi transaction; worker và batch job cũng phải có context tường minh.
- Bật ENABLE ROW LEVEL SECURITY và FORCE ROW LEVEL SECURITY trên mọi bảng tenant-scoped.
- Runtime role không phải superuser và không có BYPASSRLS. FK tenant-scoped kiểm tra cả business_id.
- Cache key, object storage path, queue message, log và kết quả báo cáo đều mang tenant identity.

Lưu ý: RLS không áp dụng cho TRUNCATE và REFERENCES, và superuser/BYPASSRLS luôn vượt policy. Vì vậy RLS là lưới an toàn cuối cùng, không thay thế quyền DB và thiết kế ứng dụng.

4. Control plane và cell

Cell là một cụm vận hành gồm gateway/service và database tương ứng. Pool cell phục vụ nhiều tenant. Dedicated cell phục vụ một tenant hoặc một nhóm tenant cần mức cách ly cao hơn.

Luồng định tuyến



Control plane lưu	Cell lưu
business_id → cell_id	Nhân viên, chi nhánh, ca làm, phiếu kết ca.
Gói dịch vụ và feature flags	Rule, workflow và field definition của tenant.
Extension binding + config version	Dữ liệu extension và audit nghiệp vụ.
Cell endpoint, health, release version	Outbox, trạng thái job và dữ liệu tích hợp.

Cách triển khai theo giai đoạn

- Hiện tại: catalog tối thiểu nằm trong platform-service; mọi tenant trở tới pool-01. Không cần một control plane service độc lập.
- Trước khi có cell thứ hai: tách rõ catalog quản trị khỏi dữ liệu nghiệp vụ và thêm cache định tuyến ở gateway.
- Khi có dedicated cell: provision cell mới, migrate dữ liệu có kiểm soát, đối soát, rồi mới chuyển mapping tenant → cell.

Mỗi service có image riêng; cùng image được dùng lại ở tất cả cell. 'Cùng image' không có nghĩa gom toàn bộ service thành một image hoặc một process.

5. Luồng dữ liệu và analytics

Luồng ghi nghiệp vụ



Ghi outbox trong cùng transaction với dữ liệu nghiệp vụ tránh trạng thái 'DB đã commit nhưng event bị mất'. Consumer phải xử lý idempotent vì event có thể được giao lại.

Không gọi workforce API, MISA hoặc dịch vụ ngoài trong transaction duyệt kết ca. Dữ liệu nội bộ cần nhất quán nên đọc từ DB chung hoặc snapshot; việc được phép trễ đi qua outbox.

Analytics theo giai đoạn

Giai đoạn	Thiết kế
Hiện tại	Schema analytics trong fnbx_oltp; view/materialized view; reader bị giới hạn theo tenant.
Tải đọc tăng	Tối ưu index/query; cân nhắc read replica cho reporting.
Nhiều cell hoặc tải phân tích ảnh hưởng OLTP	CDC/ELT fan-in về DWH/OLAP theo model chuẩn, có business_id và quyền truy cập được dựng lại tại đích.
AI / benchmark	AI đọc semantic layer có kiểm soát. Benchmark chỉ dùng dữ liệu được phép và đã tổng hợp/ẩn danh phù hợp.

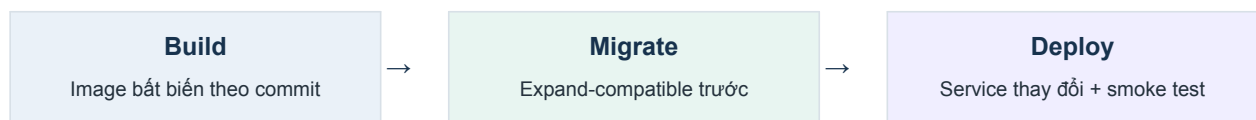
RLS của PostgreSQL không tự đi theo dữ liệu sang DWH hoặc file cột. Khi tách analytics, chính sách tenant phải được thiết kế và kiểm thử lại tại hệ thống đích.

6. Phương án deploy hợp lý nhất

Giai đoạn hiện tại: một VM ứng dụng chạy Docker Compose + PostgreSQL managed + object storage. Chưa cần Kubernetes.

Thành phần	Triển khai đề xuất
Gateway + backend	Container trên một VM Linux; chỉ chạy các service đã thực sự cần.
Console	Container trên VM ứng dụng hoặc nền tảng web phù hợp; mobile phát hành riêng.
PostgreSQL	Managed PostgreSQL, private networking, backup tự động và PITR.
PgBouncer	Transaction pooling; giới hạn connection theo service/role.
File/chứng từ	Object storage; DB chỉ lưu metadata và quyền truy cập.
Job/outbox	Worker container riêng, retry có backoff, dead-letter/alert và idempotency.
Quan sát	Log tập trung, metrics, health check và cảnh báo backup/job backlog.
Môi trường	Staging và production dùng database, secrets và storage tách biệt.

Release flow



Ở quy mô nhỏ, stop/start có graceful shutdown trong khung bảo trì là lựa chọn hợp lý. Khi cần zero downtime, tăng lên ít nhất hai replica và dùng rolling hoặc blue/green; từ thời điểm đó migration phải chịu được phiên bản cũ và mới chạy đồng thời.

Dedicated tier

Dedicated DB trên cùng một PostgreSQL instance chỉ cách ly namespace và backup tốt hơn; tenant vẫn chia sẻ CPU, RAM, I/O và miền sự cố. Nếu hợp đồng bán cách ly tài nguyên, dedicated tier phải có database compute và application capacity tương ứng.

7. Khi nào cần mở rộng

Không dùng số tenant hoặc dung lượng DB như một ngưỡng máy móc. Quyết định dựa trên chỉ số vận hành, tải giờ cao điểm và cam kết khách hàng.

Tín hiệu đã đo	Hành động
Một service quá tải	Scale replica của service đó; chưa cần tạo cell mới.
Reporting làm chậm giao dịch	Tối ưu query/index, sau đó tách tải đọc; dựng OLAP khi cần.
Tenant lớn gây noisy neighbor	Áp quota/rate limit; chuyển dedicated nếu vẫn ảnh hưởng.
Pool chạm giới hạn vận hành	Tạo pool cell thứ hai và phân nhóm tenant.
Hợp đồng yêu cầu cách ly, PITR hoặc residency	Dedicated cell theo đúng mức cam kết.
Hai nhóm phát triển độc lập hoặc CI quá chậm	Xem xét tách repo theo ownership và release cadence.

Chỉ số nên theo dõi

- p95/p99 latency và error rate theo service, tenant, endpoint.
- CPU, I/O, lock wait, connection saturation và transaction duration của PostgreSQL.
- Outbox lag, retry count, dead-letter count và độ trễ đồng bộ POS.
- Thời gian refresh báo cáo, truy vấn chậm và tác động của reporting lên giao dịch.
- RPO/RTO thực tế từ diễn tập backup-restore, không chỉ từ cấu hình nhà cung cấp.

Ngưỡng để tách phải được viết thành SLO/capacity policy và kiểm tra bằng telemetry. 'Đạt 50 tenant' tự nó không phải lý do kiến trúc.

8. Lộ trình thực thi

Ưu tiên	Việc cần làm	Kết quả mong đợi
1	Hoàn thiện deployment thật: production Compose, image registry, secrets, health check, backup và rollback.	Có thể vận hành pool-01 an toàn; tools/deploy.sh không còn là placeholder.
2	Đưa Comfy lên core chung; liệt kê mọi khác biệt và phân loại theo 6 tầng tùy biến.	Không duy trì comfy_case_close như một product fork.
3	Thêm approval rule/workflow versioning/field definition và một extension point đầu tiên.	Chứng minh A và B chạy qua cùng core.
4	Thêm tenant catalog tối thiểu; mọi tenant ban đầu map tới pool-01.	Giữ sẵn đường vào kiến trúc cell.
5	Hoàn thiện outbox, kiểm thử RLS, tenant isolation, backup và restore drill.	Đóng các rủi ro dữ liệu trước khi tăng khách hàng.
6	Khi xuất hiện nhu cầu thật, bổ sung provisioning và quy trình di chuyển tenant sang dedicated cell.	Mở tier enterprise mà không fork code.

Các quy tắc không được phá

- Không có điều kiện tenant cụ thể trong core; binding nằm trong dữ liệu.
- Không cho tenant tải code/script tùy ý chạy trong JVM dùng chung.
- Không pin phiên bản ứng dụng lâu dài theo tenant.
- Mọi custom data phải map được về model và taxonomy chuẩn phục vụ báo cáo/AI.
- Migration chạy trước deploy và theo expand → migrate → contract khi có mixed version.
- Mỗi extension có owner, test contract, telemetry và điều kiện loại bỏ hoặc nâng thành core.

Kết luận

Kiến trúc nên triển khai ngay: một pool cell, một bộ service dùng chung, một PostgreSQL managed, tùy biến bằng dữ liệu trước và extension có kiểm soát sau.

Kiến trúc đích nhiều cell là đúng hướng, nhưng không cần dựng toàn bộ từ ngày đầu. Hãy giữ các ranh giới cần thiết - tenant catalog, cell-aware routing, image bất biến, migration tương thích - rồi chỉ nhân bản cell khi telemetry hoặc hợp đồng tạo ra nhu cầu thật.

Cách tiếp cận này giữ được ba mục tiêu đồng thời: chi phí vận hành thấp khi còn nhỏ; khả năng tùy biến sâu hơn các POS phổ thông; và đường mở rộng rõ ràng cho khách hàng enterprise mà không biến sản phẩm thành tập hợp các dự án outsource.

Tài liệu tham chiếu

- [PostgreSQL 16 - Row Security Policies](#)
- [Docker Docs - Use Compose in production](#)
- [AWS Prescriptive Guidance - Transactional outbox pattern](#)
- [AWS Guidance - Cell-Based Architecture](#)

Nền tảng nội bộ được đối chiếu: README của monorepo fnb; ADR-0003 về deployment/data; ADR-0004 về analytics.